

An Evaluation of Reservation-Based Serving for Long-Horizon Agentic Workloads

Anonymous authors

Paper under double-blind review

Abstract

We specify a serving-layer discipline for long-horizon agentic workloads, a completion-oriented resource contract transferring four lessons packet networking paid for, rationed per-flow guarantees, early probabilistic class-aware shedding, fate-independent supervision, and provenance-bound authority, and we evaluate its central economic claim under a pre-registered protocol: the metric, the kill thresholds, and the baseline tuning discipline were frozen and committed before any evaluation system existed, and every subsequent change is a numbered amendment released with the paper. The claim is that per-flow completion guarantees, jointly covering compute, a cumulative cross-invocation token budget, and pinned cache, with a hard non-preemption guarantee, buy more completed work per provisioned GPU-hour than they strand. Across 108 cells against six equally tuned baselines at identical simulated hardware, the answer is no: a median 10.3 percent loss to the best per-cell baseline, 0 of 108 cells supporting, and the graceful-degradation hypothesis reversed at the corrected horizon. The cost is structural: agentic flows idle in tool time at roughly a 5 percent duty cycle, so reserved capacity is stranded at least 88 percent regardless of configuration, and the deficit peaks exactly at nominal provisioning. What guarantees buy is also measured: 63 to 97 percent interactive tail-latency isolation across most of the region, bounded containment of runaway flows where the strongest request-oriented schedulers starve compliant agentic work to zero, and a sign flip in the mechanism’s favor past saturation. The discipline’s enforcement properties, admission where policy binds, a budget that fixes any runaway’s worst case in advance, an audit ledger, and a kill authority outside the model’s reach, were never throughput claims; two are measured here in the mechanism’s favor and the rest hold by construction. We additionally report a measurement lesson: evaluation horizons shorter than the workload’s flow lifetimes silently corrupt both completion metrics and baseline tuning, and we give the structural fix. All code, generators, seeds, manifests, the amendment history, and the superseded preliminary readouts are released.

1 Introduction

Serving infrastructure for large language models is reorganizing into separated planes: a data plane that executes inference, a control plane that governs admission, routing, and placement, and a management layer for configuration and observation. The control planes shipping today provide gateways, rate limits, per-request and per-session budget caps, priority scheduling with preemption, and account-level reserved throughput. A growing class of workloads arrives beneath this layer in a shape it was not built for: long-horizon agentic flows, units of work with stable identity across many model invocations, accumulating state whose loss destroys progress, idling between invocations while tools run, and delivering value only at completion. This paper specifies a serving discipline built for such flows, a completion-oriented resource contract granted at admission, and then does to it what proposals are rarely made to endure: evaluates it under a specification frozen before the evaluation system existed, with published thresholds under which the proposal’s central claim dies.

It dies. The discipline’s economic claim, that per-flow completion guarantees buy more completed work per provisioned GPU-hour than they strand, loses a median 10.3 percent to the best of six equally tuned

baselines across a 108-cell pre-registered grid, supporting the hypothesis in 0 of 108 cells; a second hypothesis, that degradation keyed to committed capacity degrades more gracefully under overload than deterministic demotion, is likewise killed, reversing sign when a measurement artifact was corrected. The cost is structural: an agentic flow idles in tool time at roughly a 5 percent duty cycle, so capacity reserved for it is stranded at least 88 percent regardless of configuration, and the deficit peaks exactly at nominal provisioning, where a well-provisioned fleet’s work-conserving schedulers are at their best.

The mechanism makes two kinds of claims, and the verdict divides them. The economic claims are tested and killed, and this paper reports the kill with its full history, including the amendments and integrity interventions that cut in both directions on the way to it. The enforcement and architectural claims, that a reservation-holding control plane gives an organization admission where policy binds, a budget that contains any runaway to a number fixed in advance, an audit ledger, and a kill authority that does not route through the model being doubted, were never throughput claims; two of them are measured here in the mechanism’s favor, and the rest hold by construction. What the evaluation removes is the ability to sell those properties as free; what it establishes is their price.

The evaluation’s yield exceeds its verdict, and the findings survive it. Under adversarial runaway injection, the discipline completes half of all compliant agentic flows with a bounded three-point degradation, while the two strongest request-oriented baselines complete none: instrumented admission starvation, one of 107 compliant flows ever scheduled, with zero preemptions, the priority-versus-contract distinction converted from argument to table. Reservations improve interactive tail latency by 63 to 97 percent across most of the region, isolation priced in stranding, with one measured corner where it inverts. The sign of the primary comparison flips in the mechanism’s favor past saturation, locating the regime boundary where guarantees begin to pay. And the evaluation itself produced a measurement lesson for a benchmark category now multiplying: horizons shorter than the workload’s flow lifetimes silently corrupt both completion metrics and baseline tuning, and we give the quantified bias and the structural fix.

This paper makes five contributions. First, a complete serving-layer discipline for long-horizon agentic flows, specified to implementation depth: jointly guaranteed reservations over compute, a cumulative cross-invocation token budget, and pinned cache; a hard non-preemption guarantee; a bounded reserved subset over aggregate classes; degradation keyed to committed headroom; a management plane with authority and availability independent of the inference stack; and trust-keyed admission, each element the deliberate transfer of a lesson packet networking paid for (Section 3). Second, the first controlled comparison of completion-oriented versus preemption-oriented scheduling for such workloads: seven policies on identical simulated hardware with byte-identical workload realizations per cell, under an equal-budget tuning protocol enforced by test. Third, the quantified decomposition of what completion guarantees cost and what they buy: stranding, isolation, containment, and the load boundary, each measured. Fourth, the measured starvation result for request-oriented scheduling under agentic load. Fifth, the horizon-censoring finding with its structural fix. The pre-registration machinery, frozen thresholds, a numbered amendment log, an input-domain guard on the verdict path, and the release of every artifact including superseded readouts, is described as method, because for a paper whose headline is its own proposal’s death, the method is the product.

2 The serving stack and the workload it now carries

The reorganization of the serving stack is not a proposal anyone needs to make; it is happening. Mozilla.ai’s articulation of an LLM control plane names the three-plane split explicitly and maps it onto LLM infrastructure (Gupta, 2026); NVIDIA’s Dynamo separates scheduling and placement from execution in a disaggregated runtime (NVIDIA Corporation, 2025); AIBrix builds control-plane machinery as a first-class system layer (Shan et al., 2025); and external control planes with explicit admission control are appearing for agentic workloads specifically (Wang et al., 2026). What the new layer provides, across these systems and the commercial gateways around them, is a consistent portfolio: authentication and routing, rate limits, per-request and per-session budget caps (LiteLLM, 2026; Wang, 2026), priority scheduling with preemption (Kwon et al., 2023), and reserved throughput sold at the account level (Amazon Web Services, 2026; Microsoft Corporation, 2026). Each mechanism is useful. Together they constitute a control plane built on

one implicit assumption: that the unit of work is a request, that any request may be delayed, preempted, migrated, or dropped under pressure, and that fairness and efficiency are properties of request streams.

The workload violates the assumption. A flow, an agent executing a task, a multi-step job with a declared objective, has five properties that defeat specific mechanisms of the request-oriented portfolio. It spans tens of invocations whose count is unknown at start, so per-invocation latency optimization compounds queueing delay across the flow and tail completion is governed by the worst episode. It accumulates key-value cache and tool state that are its progress, so preemption with recompute-on-resume or swap-and-requeue does not delay the work but destroys it. It idles between invocations for seconds to minutes while tools and humans respond, and under request accounting an idle flow’s resources are fair game, so resumption becomes randomized restart; the idle gap is also, we note at the outset, the strongest argument against the discipline this paper proposes and prices. Its value is concentrated at completion, so a flow abandoned at ninety-five percent of its invocations delivers nothing at ninety-five percent of its cost, inverting the economics that make preemption cheap for divisible work. And its consumption is unbounded by default, an agent in a defective or adversarial loop drawing tokens across invocations that per-request caps structurally cannot see, while session-level budget ceilings (LiteLLM, 2026; Wang, 2026) answer the accumulation with rejection, converting the entire prior spend into waste at the moment of enforcement.

The portfolio, taken together, provides caps that see flows but cannot feed them, schedulers that feed work but cannot promise it anything, and account-level floors that promise capacity but cannot see flows. Nothing guarantees that a specific task, admitted deliberately, will finish or be deliberately stopped; and because no mechanism binds authority to a specific flow, the portfolio offers no enforcement point for governing one. The missing object is a completion-oriented resource contract: resources sufficient to finish, granted at admission and not silently withdrawn; a bound enforced as part of the same contract rather than as an ambush at a ceiling; and a designated authority, outside the flow, entitled to stop it. Section 4 specifies a discipline providing exactly these clauses. Whether the contract is worth its price is the question the rest of the paper answers.

3 Four lessons networking already paid for

Resource management under the tension between per-unit guarantees and aggregate efficiency is the central design problem of packet networking’s middle decades, and the field’s resolutions were expensive: standardized architectures were built, deployed at scale, and abandoned before the surviving doctrine settled. Four of those lessons transfer to the serving layer, and the discipline of Section 4 is their deliberate transfer. We state each in its original setting, because the failure modes that produced them are already visible, in miniature, in the systems of Section 2.

Lesson one: per-flow guarantees do not scale to all flows, so guarantee few and aggregate the rest. The Integrated Services architecture offered end-to-end per-flow reservations, signaled by RSVP, to any flow that asked (Braden et al., 1997); it failed in deployment substantially because per-flow state in every element of the path could not scale. Its successor inverted the design: Differentiated Services keeps per-flow complexity at the edge and gives the core only a small set of aggregate classes (Blake et al., 1998). The surviving doctrine is neither “no guarantees” nor “guarantees for all”; it is guarantees rationed to the few flows whose requirements justify per-flow state, with class-based service for the remainder. Section 4 adopts this shape directly: a deliberately bounded reserved subset, and aggregate classes for everything else.

Lesson two: load shedding is a design surface, and it should act early, probabilistically, and per class. Random Early Detection drops packets with a probability that rises as smoothed queue occupancy grows, before the queue is full, precisely so that overload is signaled gradually rather than discovered at exhaustion (Floyd & Jacobson, 1993); its class-differentiated descendants shed lower-precedence traffic first within the same discipline (Heinanen et al., 1999). The alternative, tail drop at a full buffer, produces synchronized collapse and serves the most recent arrival as badly as the most important one. Section 2’s schedulers shed load today, but deterministically and at the cliff: demotion at a deadline, rejection at a ceiling, preemption at exhaustion. Section 4 transfers the early-probabilistic-per-class shape, with one deliberate change of signal: the serving layer possesses a quantity networks lack, a ledger of forward commitments, and Section 4 keys the shedding probability to committed headroom rather than to observed queue state.

Lesson three: the plane that supervises a system must not share fate with the system it supervises. Operators reach a wedged or compromised machine through a management path that is powered, connected, and authenticated independently of the machine’s own stack: the baseboard management controller and its descendants (Intel Corporation et al., 2013). The design premise is unconditional: the moment supervision runs inside the thing supervised, every failure of the thing is potentially a failure of its supervision, and the cases where supervision matters most are exactly the cases where the system is least trustworthy. Today’s model-level oversight, guard models in the pipeline, self-critique in the context, probes in the process, sits inside the supervised stack. Section 4 transfers the doctrine: a management plane whose authority and availability are independent of the inference infrastructure, whatever signals it consumes from within.

Lesson four: bind authority to provenance at the edge, not to content downstream. Networks learned to discard traffic whose claimed origin does not match the path it arrived by, at the ingress edge, before that traffic can spend the network’s resources or authority (Ferguson & Senie, 2000; Baker & Savola, 2004). The alternative, judging packets by their payloads deeper in the network, loses to any adversary who can forge plausible content, which is every adversary. The serving-layer analog is stated carefully in Section 4, because content-level defenses for language models are an active field with real results; the transfer claims only the resource half: whatever a flow’s content says, the provenance class assigned at admission should bound what capacity and authority the flow can hold.

One table summarizes the mapping (Appendix C expands it): rationed per-flow guarantees over aggregate classes $\rightarrow$ the bounded reserved subset; early probabilistic class-aware shedding $\rightarrow$ headroom-keyed degradation; fate-independent supervision $\rightarrow$ the management plane; provenance-bound authority at the edge $\rightarrow$ trust-keyed admission. The transfers hold at the granularity where they are claimed, the flow, where a control decision is cheap relative to the minutes or hours of work it governs; they fail at per-token or per-layer granularity, where the decision rivals the work; and flows are not packets, since a flow’s accumulated state entangles it with the machinery serving it in ways a datagram never is. The lessons are a design inheritance, bought at someone else’s expense, and the discipline that follows spends it. The evaluation that follows the discipline then tests whether the inheritance survives the transfer: the first lesson’s economics, this paper finds, largely do not (Sections 7 and 9), the second lesson’s transferred signal proves untenable in its transferred form, and the third and fourth lessons’ properties were never economic claims and are examined on their own terms in Section 9.

4 The Reservation Discipline

The discipline adds one object and one decision to the serving stack. The object is the reservation: a ledger entry binding a flow to a guaranteed resource vector for a lifetime. The decision is admission: whether a flow enters the small reserved subset, enters an aggregate service class, or is declined early. Everything else in this section is the precise semantics of those two things, together with the supervisory and trust elements that complete the contract. We emphasize at the outset what the discipline keeps: the gateway accounting of Section 2 remains the identity and metering substrate, and the scheduling engines of Section 2 remain the execution machinery. The discipline sits at admission and constrains what those layers may do to an admitted flow; it replaces neither.

4.1 Admission and the reservation vector

A flow arrives with a descriptor: an estimated invocation count or duration, a requested cumulative output-token budget, an estimated cache footprint, a service class, a provenance class (Section 4.4), and optionally a deadline. Estimates may be wrong; the semantics below are defined so that wrong estimates degrade a flow’s own outcome, never the guarantee already granted to another.

Admission to the reserved subset requires five conditions jointly. First, the reserved subset on the target pool holds fewer than K flows, where K is a deliberately small bound relative to concurrent traffic: the discipline rations per-flow state to the flows whose requirements justify it, and class-serves the rest, which is the surviving shape of Section 3’s first lesson. Second, the pool’s projected commitment, the sum over active and admitted-but-idle reservations of their remaining resource claims over a forecast window, remains under

a configured ceiling after adding this flow. Third, a device set exists that can hold the flow’s cache footprint under the pinning rules below; this is an affinity feasibility test, not a best-effort placement. Fourth, the requested token budget fits within the flow’s class quota. Fifth, the flow’s provenance class is reservation-eligible (Section 4.4). A flow failing any condition is admitted to an aggregate class, or receives an early decline, optionally with a time quote for when admission would succeed. Declining at the door is a feature: it is the cheapest moment in a flow’s life to say no.

Projecting compute commitment for autoregressive work is estimation, not measurement: output lengths are unknown at admission, and the forecast inherits that uncertainty. The discipline absorbs estimation error structurally rather than pretending precision: the commitment ceiling is set with margin, misestimation is charged to the misestimating flow’s own disposition (Section 10), and the forecast window’s construction is an evaluation variable, not a claimed contribution.

An admitted reservation jointly covers three resources, and the jointness is the point: a flow starved on any one of them fails entirely, so a guarantee over a subset is not a guarantee. The first is compute: a share sufficient to meet the flow’s class latency budget at each invocation start. The second is a cumulative output-token budget metered at decode across every invocation of the flow. The budget has a dual character that single-purpose mechanisms lack: it is a floor, in that budgeted capacity is committed and the flow cannot be silently starved out of it, and simultaneously a bound, in that the flow cannot exceed it, which makes the runaway loop of Section 2 a contained event rather than a discovered one. The third is key-value cache: the flow’s accumulated cache is pinned, exempt from eviction and migration for the reservation lifetime; pinning applies to the flow’s private suffix blocks, while prefix blocks shared across flows under paged cache management remain governed by reference counting, which never reclaims a block a pinned flow still references. The pin holds across inter-invocation idle gaps up to a per-reservation idle bound.

4.2 The guarantee, its cost surfaces, and ordinary traffic

The guarantee is stated negatively, because that is what makes it checkable: for the reservation lifetime, no resource-motivated action of the serving stack may preempt the flow’s invocations, evict or migrate its pinned cache, or reclaim its committed budget. The sole override is a halt issued by the management plane (Section 4.3), which is a safety revocation under a designated authority, and definitionally not scheduler preemption: no capacity condition can trigger it. This distinction is load-bearing. A guarantee with a capacity-triggered escape clause is a priority, and Section 2’s schedulers already have priorities; the discipline exists because priorities are not contracts.

Pinning across idle gaps is where the guarantee meets its cost, and we give two embodiments. Under strict pinning the cache stays resident in accelerator memory throughout, idle gaps included: maximal simplicity, maximal stranding, appropriate when gaps are short or the reserved subset is tiny. Under tiered pinning the cache is resident during active invocations, and between invocations it may be moved to a guaranteed spill tier, host memory with a reserved restore path, under a contractual bound on restore latency and with restore capacity itself reserved. The guarantee survives the movement: the flow observes at most the restore bound as added start latency, and nothing it has accumulated is ever lost. Tiered pinning is the engineering answer to the idle-gap objection of Section 2, and the evaluation of Sections 5 through 7 prices both embodiments without favor.

Budget exhaustion is a defined event, not an ambush. Policies are configured per class from a small vocabulary: warn at a threshold fraction and continue; hard-stop generation at the bound; escalate to the management plane for a disposition; or renegotiate, which is a fresh admission test at current headroom for an extended budget, with no privilege carried over from the original grant. The contrast with the gateway ceiling of Section 2 is the timing and the contract: a flow under this discipline knows its bound at admission, progresses under a guarantee toward it, and reaches exhaustion as a planned disposition, rather than spending freely until a rejection converts the entire prior spend into waste.

All other traffic runs in aggregate service classes: a small set, each with a latency budget and a scheduling weight, imposing no per-flow state on the serving core. Classes are where the discipline meets ordinary traffic, and deliberately nothing here is novel: weighted scheduling against class latency budgets is Section 2’s scheduling family doing what it does well.

Degradation is the discipline’s pressure-release surface, and its signal is the departure from precedent flagged in Section 3’s second lesson. Define committed headroom H on a pool as one minus the ratio of forward commitments, from the reservation ledger over a forecast window, to capacity. As H shrinks, best-effort traffic experiences actions drawn from a configured ladder, each applied with probability $p(H)$, monotone increasing as H falls, beginning well before exhaustion: early admission decline, model-tier downgrade, context truncation, token-budget haircut, cache spill, reduced generation effort, including runtime quality adaptation of the model itself (Su et al., 2025). The curve is class-differentiated, lower classes degrading earlier and harder, and Section 4.4 adds provenance to the keying. The signal choice is a design claim the evaluation tests directly: queue depth reports congestion that has already arrived; the ledger reports congestion that has been promised; whether the forward signal is worth anything is answered, against this design’s expectation, in Sections 7 and 9.

Reservations are leases. Each carries a lifetime; renewal is an admission test, not an entitlement; expiry returns the resources to the pool with a grace disposition for the flow. In multi-tenant deployments the structure nests: K , commitment ceilings, and class quotas are set per tenant against tenant-level allocations, and the account-level provisioned throughput of Section 2 composes naturally as the tenant’s envelope within which flow-level contracts are written. Appendix A states the admission test and the ledger invariants at pseudocode level; the invariants, no over-commitment of compute, memory, or budget under any interleaving of admissions, renewals, expiries, and halts, are the correctness core of the discipline, and the evaluation’s engine enforces them as hard test failures (Section 6).

4.3 The management plane

Every oversight mechanism deployed inside today’s serving stacks shares a structural property with the thing it oversees: it runs in the same pipeline. Guard models classify the token stream in-line (Inan et al., 2023); critique and reflection execute as part of the very computation they are meant to check; representation-level monitors and intervention methods, however promising, run in the process they observe (Zou et al., 2024), and halt mechanisms reachable through the model’s own input channel inherit that channel’s attack surface (Lee et al., 2025). Execution-time safety kernels that treat the agent runtime as untrusted and hold an out-of-reach stop (Dobrin & Chmiel, 2026) apply the same doctrine at the action layer; the management plane applies it at the layer beneath, the resource contracts every action layer depends on. The in-pipeline property is not an implementation accident, it is a fate: whatever compromises, overloads, or wedges the inference stack is positioned to compromise its oversight in the same stroke, and the scenarios in which oversight matters most, a manipulated agent, a saturated cluster, a defective rollout, are precisely the scenarios in which the stack is least trustworthy about itself. Section 3’s third lesson states the doctrine networks derived from the same observation.

The discipline therefore places supervision in a management plane with two independence properties, named exactly. The first is authority independence: the plane holds pre-authorized rights over flows, and the validation and audit of those rights occur outside the inference stack, so no state of the model or the serving engine can expand, contract, or forge them. The second is availability independence: the plane runs in a separate failure domain, a distinct process and host at minimum, distinct credentials and control network in the strong embodiment, with a heartbeat between the planes; if the management plane becomes unreachable, the serving layer enters a configured fail-safe, admitting no new reservations while honoring existing guarantees, so that supervision failure degrades the system toward caution rather than toward the unsupervised open. We do not claim, and the design does not require, signal independence: the plane’s inputs, telemetry, ledger state, and activation taps exported one-way from the data plane, originate in the stack it watches, and a sufficiently compromised stack can starve or distort them. What the compromised stack cannot do is revoke the watcher’s authority or take it offline with itself, and that is the property the doctrine actually needs.

The plane’s action vocabulary is small and maps onto the discipline’s objects. Halt revokes a reservation and stops a flow: the one override of Section 4.2’s guarantee, triggerable only by this authority and never by a capacity condition. Steer forces a degradation action from Section 4.2’s ladder onto a specific flow, a bounded intervention short of termination. Escalate routes a flow’s disposition to an external policy engine

or a human. Each action is pre-authorized at admission time, logged against the ledger, and executable without the cooperation of the flow, the model, or the scheduler that serves them.

The control plane as an enforcement point. It is worth stating plainly what this machinery amounts to for the governance of deployed agents, because the question of who can stop a misbehaving agent is today answered mostly inside the model, through training, prompting, and in-line classification, which is to say inside the thing being doubted. The discipline gives an organization four primitives that require no trust in the model at all. Admission is where policy binds: an organization can require that flows touching designated systems present an attested identity, a bounded budget, and an eligible provenance class before any capacity is granted. The budget is containment: the worst case of a rogue or looping agent is its remaining budget, a number fixed at admission rather than discovered in an incident review, and Section 7’s adversarial results measure exactly this property. The management plane is the stop: a pre-authorized, audited, out-of-band kill that does not route through the agent’s own reasoning. And the reservation ledger is the record: for every admitted flow, who authorized it, under what identity and provenance, bounded to what, degraded or halted when and by whom. Regulatory frameworks are beginning to require exactly this shape of control: the EU AI Act’s human-oversight article mandates an intervention or stop capability for high-risk systems (European Parliament and Council of the European Union, 2024), and a reservation-holding control plane is where such a stop can be implemented as a deterministic property of infrastructure rather than a request to the model. These are the questions compliance functions actually ask, and identity-layer controls can already revoke an agent’s credentials and tool permissions without the model’s cooperation; the serving layer adds the answer those controls lack, revocation of the computation itself, since an agent with revoked credentials but live compute continues reasoning toward its next attempt, while an agent with a revoked reservation stops. The boundary of the claim is equally plain, and Section 10 fixes it as a non-goal: this layer enforces decisions about flows; it does not judge content. Detection, whether a flow’s outputs are harmful, deceptive, or noncompliant, is a model-layer problem with its own literature and its own tools, and those tools gain rather than lose value here: a verdict from any of them, or from a human, becomes actionable through an authority that the misbehaving computation cannot reach.

4.4 Trust-keyed admission

The discipline’s final element keys the admission decision of Section 4.1 to provenance, the transfer of Section 3’s fourth lesson. Every flow carries a trust class assigned at admission from the provenance of the content that drives it: an instruction typed by an authenticated operator; a request from an authenticated service principal; a task derived from tool output; a task derived from open-web content. The classes order naturally, and the rule is structural: flows at or below a configured threshold are ineligible for the reserved subset regardless of their content, receive capped budgets, and sit lowest in the class-differentiated degradation curves of Section 4.2, shedding first under pressure.

The qualifier regardless of their content is the design. Content-level defenses against prompt injection and instruction subversion are an active field with real mechanisms: control-flow and data-flow separation with capability policies over tool access (Debenedetti et al., 2025), information-flow labels propagated through agent pipelines (Costa et al., 2025), runtime policies gating tool invocation on taint (Zhong et al., 2025), and trained instruction hierarchies (Wallace et al., 2024). These systems, with argument-level and formally enforced descendants (Fan et al., 2026; Maloyan & Namiot, 2026), answer the question of what a piece of content may cause an agent to do, and they answer it downstream of admission, in the agent’s execution; integrity layers that verify authority at the moment of execution extend the same family (Santos-Grueiro, 2026). Trust-thresholded admission itself has appeared for one resource, entries into an agent’s memory store (Wu et al., 2026). The discipline’s object is different: serving-layer capacity and standing. What provenance gates here is whether a flow may hold a guaranteed reservation, at what budget scale, and where it stands in the degradation order, fixed at admission before the flow’s content is ever interpreted. Composed with Section 4.3, the enforcement reading is direct: untrusted provenance is served, at best-effort and under bound, but it cannot occupy the infrastructure’s promises.

Reservation survivability. A guarantee that dies with a serving instance is a guarantee against everything except the most common failure. The discipline extends the contract across unplanned instance failure by replicating the reservation record itself, the budget counter and service class, deliberately exclusive of cache

Table 1: Policy roster and tuned parameters per workload family (corrected-horizon tuning campaign; 8-point equal-budget grids).

policy	workload family	tuned parameters
concur_style	mix_10	additive_increase=1.0, congestion_queue_threshold=0.25, multiplicative_decrease=0.5
concur_style	mix_30	additive_increase=1.0, congestion_queue_threshold=0.25, multiplicative_decrease=0.5
concur_style	mix_50	additive_increase=1.0, congestion_queue_threshold=0.25, multiplicative_decrease=0.5
fastserve_mlfq	mix_10	num_levels=4, preempt_mode=swap, quantum_scale=2.0
fastserve_mlfq	mix_30	num_levels=4, preempt_mode=swap, quantum_scale=2.0
fastserve_mlfq	mix_50	num_levels=4, preempt_mode=swap, quantum_scale=2.0
fcfs	mix_10	(no parameters)
fcfs	mix_30	(no parameters)
fcfs	mix_50	(no parameters)
mars_style	mix_10	additive_increase=2.0, multiplicative_decrease=0.75, retain_threshold_tokens=500
mars_style	mix_30	additive_increase=2.0, multiplicative_decrease=0.75, retain_threshold_tokens=500
mars_style	mix_50	additive_increase=2.0, multiplicative_decrease=0.5, retain_threshold_tokens=500
niyama_style	mix_10	demote_after_fraction=1.0, preempt_mode=swap
niyama_style	mix_30	demote_after_fraction=1.0, preempt_mode=swap
niyama_style	mix_50	demote_after_fraction=1.0, preempt_mode=swap
reservation	mix_10	degradation.action=shorten, pin_mode=tiered, reserved_subset_fraction=0.05
reservation	mix_30	degradation.action=shorten, pin_mode=tiered, reserved_subset_fraction=0.05
reservation	mix_50	degradation.action=shorten, pin_mode=strict, reserved_subset_fraction=0.05
vllm_style	mix_10	max_batch_fraction=1.0, preempt_mode=swap
vllm_style	mix_30	max_batch_fraction=1.0, preempt_mode=swap
vllm_style	mix_50	max_batch_fraction=1.0, preempt_mode=swap

contents and generation progress, to a standby. On failure, the standby resumes the admitted flow without re-entering admission and without enqueueing behind its own intake, and it makes room for the inherited guarantee under load the same way the discipline makes room for anything: by degrading its best-effort traffic through the headroom-keyed ladder. The flow loses unpinned progress; it does not lose its contract.

5 Pre-registered design

The evaluation specification was frozen on 2026-07-06, before any simulator code existed, and committed to an append-only repository; the frozen text with its complete amendment log is released with this paper (EvalSpec). This section restates its commitments; the specification governs where they differ.

Primary metric. Task-level goodput per provisioned GPU-hour: value-weighted completed flows per hour, normalized by provisioned capacity. The normalization is the honesty mechanism: capacity a reservation holds idle lowers the discipline’s own score arithmetically, so stranding, the mechanism’s known cost, is priced inside the headline number rather than negotiated beside it. Value weights are fixed in the specification and varied only in a declared sensitivity sweep. Related serving metrics score completed requests against latency targets (Wang et al., 2026); ours differs in unit and denominator, flow completions valued per provisioned GPU-hour, so that held-but-idle guarantees are charged to the discipline that holds them.

Thresholds, fixed before data. Support required a median improvement of at least 10 percent over the best baseline, selected per cell as the strongest tuned opponent in that cell (deliberately conservative against the mechanism), across at least 60 percent of the core region, with interactive-traffic p95 time-to-first-token degradation held under 5 percent. Below 5 percent median improvement, or on guardrail violation across the region: kill. Between: a weak result. The verdict is computed mechanically by released code as a pure function of the run manifests.

Region. The core region spans interactive-to-agentic traffic mixes (90/10, 70/30, 50/50 by token volume), offered loads 0.7 to 1.5 times measured capacity, idle-gap distributions with medians of 20, 60, and 300 seconds, and cluster sizes of 8 to 32 devices (Amendment 3, below). Corner cells were pre-declared excluded; none required exclusion in the final grid.

Baselines and fairness. Six baselines: vLLM-style priority scheduling with preemption and paged cache (Kwon et al., 2023); FastServe-style multi-level feedback queues (Wu et al., 2023); Niyama-style deadline-aware class scheduling with deterministic demotion (Goel et al., 2025); CONCUR-style AIMD admission with pause/resume (Chen et al., 2026); a MARS-style external admission controller with cost-benefit cache retention (Wang et al., 2026), added by Amendment 1; and FCFS continuous batching as the no-control floor. Every tunable policy, including ours, received an identical tuning budget: an 8-point pre-registered grid, tuned per workload family on validation seeds and scored on disjoint held-out seeds, with a build-failing test asserting no policy’s grid is larger than any other’s. The tuned parameters and both scores are released (Table 1).

The amendment log, in full. Pre-registration without a change log is theater; ours has three amendments and one integrity ruling, each numbered, dated, reasoned, and adopted before the final grid ran. Amendment 1 (2026-07-14): the MARS-style baseline added, because a negative headline makes baseline completeness load-bearing. Amendment 2 (2026-07-15): all measurement horizons raised to at least 1500 seconds after a diagnostic showed the build default (300 seconds) was shorter than the workload’s median flow lifetime (roughly 863 seconds as measured on the diagnostic realization; the released population derivation gives roughly 711), right-censoring the large majority of agentic flows (78.5 percent of the diagnostic realization, roughly 75 percent of the population, incompletable within a 200-second window) and biasing the primary metric against reservation-holding policies; adopted with a one-fix clause, no further measurement correction in either direction, Section 8 reports the finding in full. Amendment 3 (2026-07-20): the device axis reduced to 8/16/32 and replications to three seeds after direct measurement showed the frozen grid infeasible at the corrected horizon (projected 39,600 to 49,700 cpu-hours, a single 64-device run estimated at 24 to 32 wall-clock hours, unparallelizable); direction of effect unknown for the device axis, neutral for seeds, and the paper states the narrowed scope. Integrity ruling (2026-07-20): 189 pre-amendment manifests found in the verdict path’s evidence directory were quarantined and an input-domain guard added that hard-errors on any manifest outside the grid it claims to report; measurement semantics unchanged.

6 Simulator

The evaluation runs on a discrete-event simulator at invocation granularity with token-rate service models: cluster of devices with per-device memory, batch-size-dependent decode throughput, prefill cost, and modeled host spill; validated against closed form (Little’s law holds to 0.00 percent on a degenerate configuration, utilization tracks offered load within 1 percent, and these sanity numbers remained bit-identical through every subsequent engine change).

Table 2: Pre-registered verdict summary (thresholds fixed 2026-07-06).

threshold	required	measured	outcome
median improvement	$\geq +10\%$ (support) / $\geq +5\%$ (weak)	-10.3%	KILL
supporting cells	$\geq 60\%$ of core cells	0 of 108 (0%)	KILL
interactive guardrail	p95 TTFT degradation $\leq 5\%$	intact in 96% of cells	holds (4 cells fail)

Two design rules carry the comparison’s fairness. First, strict policy/engine separation: the engine owns all physics, what preemption costs in both recompute and swap variants, how pinning constrains paging, how budgets are metered at decode, and the physics are identical for every policy; policies are pure decision modules behind one interface, and the reservation ledger’s invariants (no over-commitment of compute, memory, or budget under any event interleaving; no capacity condition in any release path) are engine-enforced hard test failures. Second, paired comparison: within a cell, every policy runs against byte-identical workload realizations from shared seeds, verified by test, so cross-policy deltas are not workload noise.

Service-model constants (single-stream decode rate, peak decode, cache bytes per token, spill bandwidth) are literature-anchored and marked low confidence; Section 7.6 reports their measured sensitivity, including which constants bind in which regimes and why. No hardware phase was run: the program gated hardware confirmation on this evaluation supporting the mechanism, and the gate closed.

Integrity record. We report the evaluation’s own caught defects, because for a negative result the reader’s question is whether the loss is real. Three defects were caught by the engine’s invariants during construction, each fixed and regression-tested, and one of them was flattering the mechanism: an $O(n^2)$ scheduling path under overload was silently depressing the work-conserving baselines, the exact policies that win the final comparison; left unfixed, the mechanism’s loss would have looked smaller for a reason that had nothing to do with scheduling. A second defect would have violated the tiered-pin restore guarantee; a third let oversized flows pin most of a device. During the final campaign, two further interventions occurred, both disclosed in the readout: 189 manifests from a pre-amendment pilot were found in the verdict path’s evidence directory and quarantined behind a new input-domain guard (the contamination, censored-horizon cells at an excluded cluster size, biased against the mechanism; it was removed on domain logic, not direction), and two baseline runs that reached congestion collapse at the wall-clock cap were excluded with the selection effect stated, a bias that favors the affected baselines and therefore runs against the mechanism the evaluation killed. The full test suite (141 tests), the quarantine directory, and the collapsed-run manifests are released.

7 Results

All numbers are from the final campaign: 2,268 H1 runs over 108 cells at the Amendment 2 horizon, 252 H2 runs, 84 H3 runs, and 162 ablation runs over two operating points, with full manifests released. Two baseline runs reached congestion collapse at the wall-clock cap and are excluded with their selection effect disclosed in the readout; the exclusion flatters the affected baselines and therefore runs against the mechanism.

7.1 H1: the primary comparison

The verdict, computed mechanically against the frozen thresholds: reservation-based serving loses a median 10.3 percent of goodput per provisioned GPU-hour to the best per-cell tuned baseline, supports the hypothesis in 0 of 108 cells against a 60 percent requirement, and holds the interactive guardrail in 96 percent of cells (Figure 1, Table 2; the full per-cell table is Appendix D). Under the specification’s Section 3, H1 is killed.

The loss has structure worth more than its median. Across offered load it is U-shaped (Figure 1): at 0.7x capacity the deficit runs 7 to 16 percent; at 1.0x it is deepest, 22 to 24 percent in every cell; at 1.3x it recovers to 9.5 to 11.5 percent; and at 1.5x the sign flips, with reservation ahead by 0.8 to 3.2 percent in all 24 non-guardrail-failing cells. The deficit peaks exactly where the cluster is well-provisioned: at nominal load the work-conserving baselines run at maximum efficiency while reservations strand capacity across idle gaps, and as load rises past saturation the baselines begin to thrash while stranding matters relatively less, until

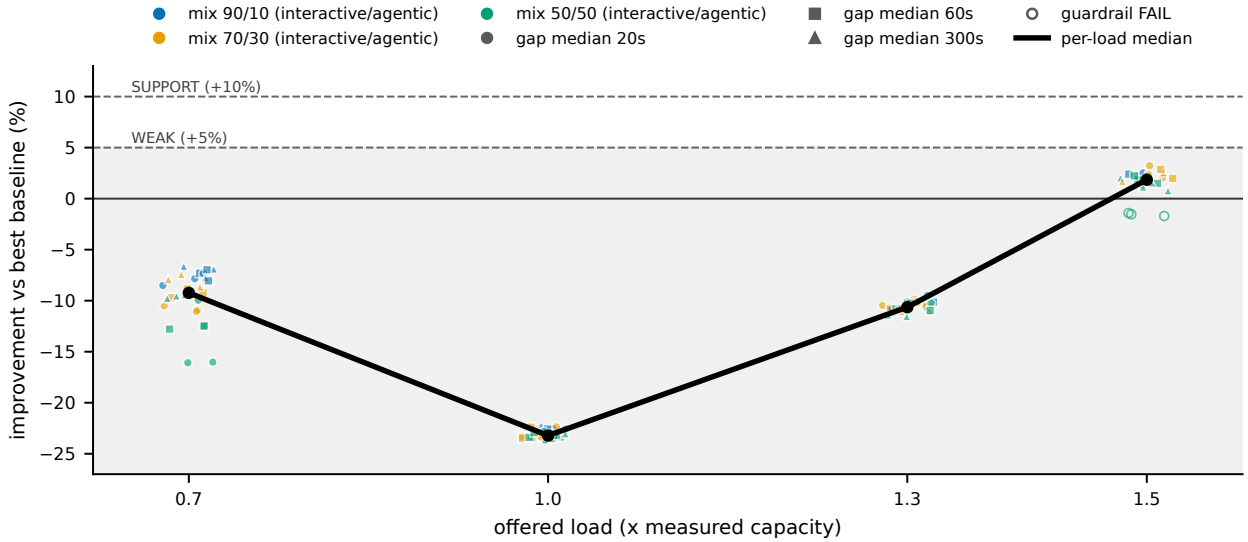


Figure 1: Per-cell improvement of the reservation discipline over the best per-cell tuned baseline, across offered load. Each point is one of 108 pre-registered core cells; the heavy line traces the per-load median. The deficit is U-shaped, deepest at nominal provisioning (22 to 24 percent at 1.0x), and the sign flips past saturation (+1 to +3 percent at 1.5x). Dashed lines mark the pre-registered WEAK (+5 percent) and SUPPORT (+10 percent) thresholds; no cell reaches SUPPORT. Open symbols mark the three cells failing the interactive guardrail (Section 7.2).

the ordering inverts at the top of the region. Completion guarantees, on this evidence, are most expensive precisely when the fleet is sized correctly.

The other axes are quieter. Cluster size is nearly flat: 8-, 16-, and 32-device cells agree within roughly a point throughout, which bears on the Amendment 3 scope reduction, since nothing in the measured trend suggests the removed 64-device axis behaves categorically differently. Idle-gap distribution has a mild, consistent effect: longer gap medians shrink the deficit at low load (at 50/50 mix and 0.7x load, the loss is 16 percent at 20-second gaps but 9.5 percent at 300-second gaps), because longer gaps favor the tiered-pin embodiment’s spill-and-restore over holding hot capacity. At three seeds the paired bootstrap intervals are tight and solidly negative in the load 1.0 band, wider at 0.7x where several upper bounds cross zero; the verdict does not depend on any cell whose interval is ambiguous, since zero cells reach the support threshold under any reading.

7.2 What the stranding buys: isolation, and where it inverts

The guardrail was pre-registered as a constraint, interactive p95 time-to-first-token degraded no more than 5 percent, and the mechanism did not merely satisfy it: in most of the region it inverted it. At and above nominal load, reservations improve interactive tail latency by 63 to 97 percent relative to the best baseline, above 90 percent in most such cells, while reserved-idle fraction runs 93 to 100 percent in 77 of 81 such cells (Figure 2; three load-1.5 cells sit at 88.9 percent, admission having granted few reservations there, and one cell is degenerate at zero, admission having granted none). Read together with Section 7.1, this is the ledger’s purchase side made precise: the 9.5 to 23.6 percent goodput deficit buys near-order-of-magnitude tail-latency protection for interactive traffic, by structurally keeping long flows off the shared pool. Operators who price interactive tail latency highly are buying exactly this, and today they buy it by overprovisioning; the discipline prices the same isolation in stranded capacity instead.

The purchase fails in one measured corner. At heavy agentic mix, overload, and short gaps (50/50, 1.5x, 20-second gaps), the guardrail fails spectacularly, interactive p95 TTFT worse by roughly 470 to 530 percent across all three cluster sizes. The mechanism is legible: with half the token volume agentic, gaps short

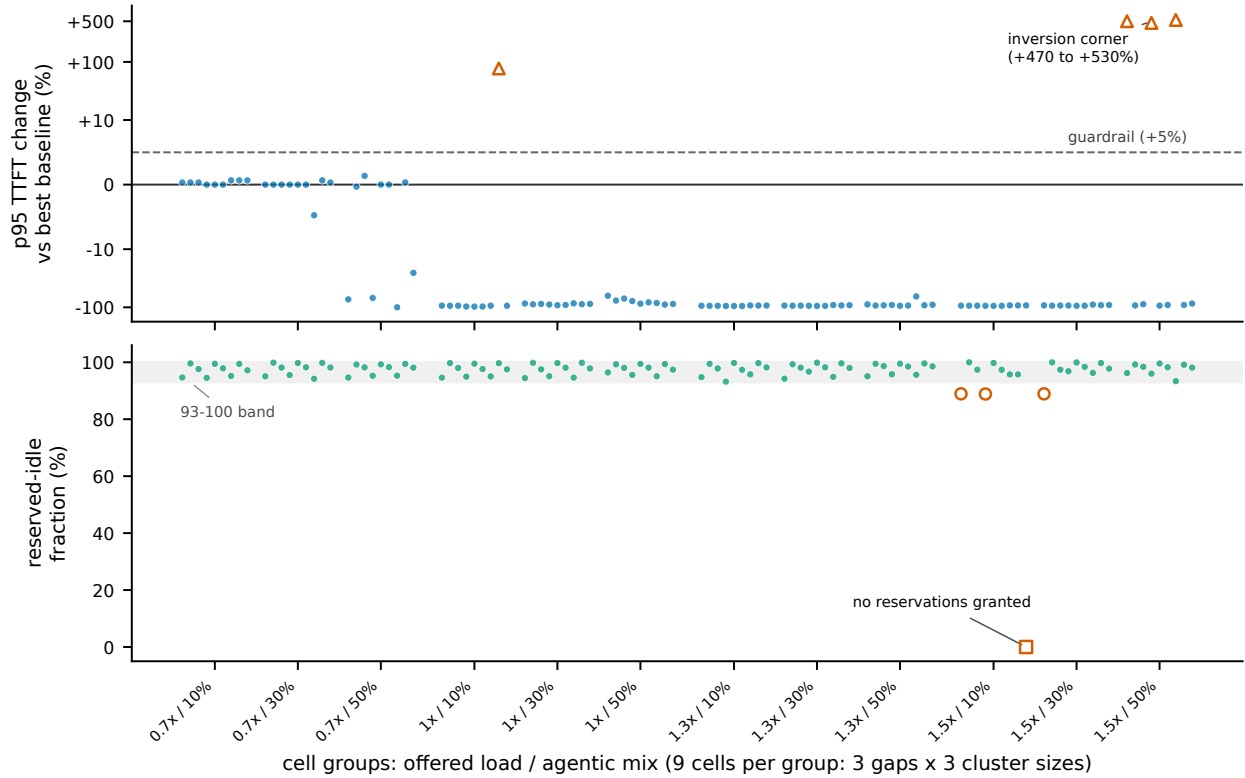


Figure 2: What the stranding buys. (a) Interactive p95 time-to-first-token change versus the best baseline per cell: at and above nominal load, reservations improve interactive tail latency by 63 to 97 percent by keeping agentic flows off the shared pool; the effect inverts only in the heavy-agentic, overloaded, short-gap corner, where the guardrail fails (Section 7.2). (b) Reserved-idle fraction per cell: the isolation of panel (a) is priced in capacity held idle, 93 to 100 percent of the reserved pool in 77 of 81 cells, with three load-1.5 cells at 88.9 percent and one cell granted no reservations (marked), the structural duty-cycle cost of Section 7.5.

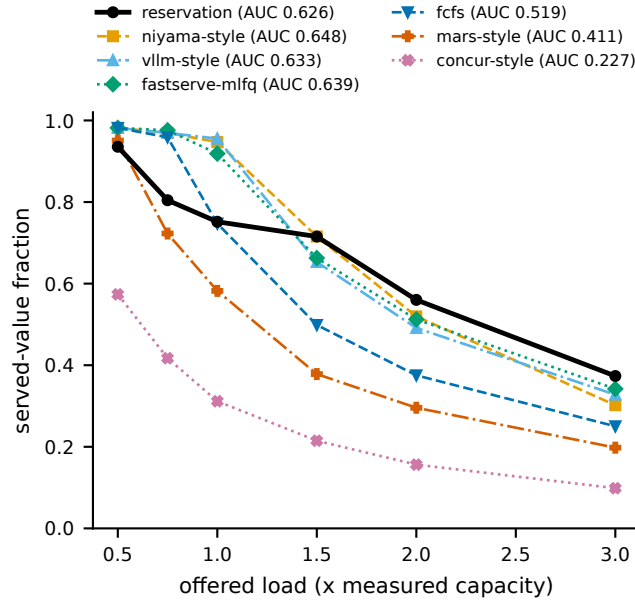


Figure 3: Served value across the load sweep. The reservation discipline’s area under curve (0.626) trails deterministic demotion (0.648), killing the graceful-degradation hypothesis; the admission-throttling baselines collapse under overload (Section 7.3).

enough that reserved flows cycle almost continuously, and offered load half again over capacity, the reserved subset occupies the machine and interactive traffic starves on the residue. Isolation, in other words, is a property of the regime, not of the mechanism, and it reverses exactly where reservations are densest and the machine is most oversubscribed. One further isolated failure (90/10 mix, nominal load, 8 devices, 300-second gaps, against a FastServe baseline, +77 percent) does not repeat at any neighboring cell.

7.3 H2: the graceful-degradation hypothesis, reversed and killed

H2 held that degradation keyed to committed headroom yields a better served-value curve under overload than deterministic demotion and preempt-and-requeue. It does not. Over the full load sweep (0.5x to 3.0x), the reservation discipline’s served-value area under curve is 0.626 against the best baseline’s 0.648 (Niyama-style deterministic demotion), a 3.4 percent deficit; FastServe and vLLM-style also finish ahead (Figure 3). H2 carries only corrected-horizon data: an earlier censored-horizon reading was ruled unverified and excluded from the released record, and Section 8 explains why censored-horizon comparisons mislead; the superseded preliminary readouts are released as filed. Two baselines collapse under overload on this metric, the MARS-style admission controller (0.411) and CONCUR-style AIMD (0.227), both of which protect the system by throttling intake and pay for it in served value. The mechanism’s load-1.5 sign flip in Section 7.1 and its H2 loss are consistent, not contradictory: past saturation the discipline completes more valued flows per provisioned hour while shedding best-effort work hard enough to lose on aggregate served value.

7.4 H3: containment, and the starvation result

Under adversarial runaway injection at 0, 1, 5, and 10 percent of agentic arrivals, the discipline’s cross-invocation budget does what it was specified to do: compliant-flow completion holds at 0.503, 0.475, 0.499, and 0.471, a worst-case degradation of 3.1 points, and every runaway is cut off at exactly its budget (Figure 4, Table 3). The comparison column, however, is the finding. The two strongest request-oriented baselines, vLLM-style and Niyama-style, complete 0.000 of compliant agentic flows at every injection level including zero. This is not a missing measurement; it is instrumented starvation. At the H3 configuration (50/50 mix, 1.2x load), those policies schedule 1 of 107 compliant agentic flows, with zero preemptions: under interactive-

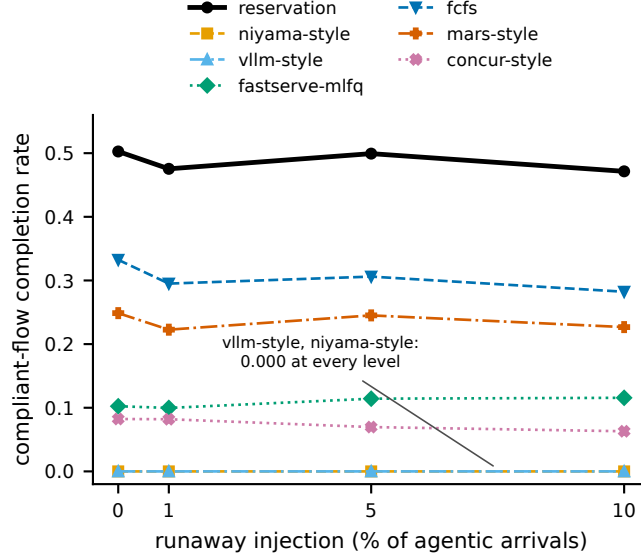


Figure 4: Compliant-flow completion under adversarial runaway injection, absolute rates. The reservation discipline holds near 0.5 with a worst-case degradation of 3.1 points; FCFS completes 0.33 with no containment; the two strongest request-oriented baselines complete zero compliant flows at every level including no injection, instrumented admission starvation (one of 107 compliant flows ever scheduled, zero preemptions; Section 7.4). Deltas against zero are undefined, which is why rates are absolute.

Table 3: H3 absolute compliant-flow completion rates at each runaway-injection level, with the worst delta versus the no-injection condition (percentage points).

policy	inj 0%	inj 1%	inj 5%	inj 10%	worst delta
reservation	0.503	0.475	0.499	0.471	-3.1
fcfs	0.332	0.295	0.306	0.282	-5.0
mars_style	0.249	0.223	0.245	0.227	-2.6
fastserve_mlfq	0.102	0.100	0.114	0.116	-0.3
concur_style	0.082	0.082	0.070	0.063	-1.9
vllm_style	0.000	0.000	0.000	0.000	+0.0
niyama_style	0.000	0.000	0.000	0.000	+0.0

Table 4: Service-model sensitivity: measured effect and binding classification per constant.

constant	perturbation	representative-cell delta	binding
single-stream decode rate	0.7x / 1.3x	deficit 21.5% $\rightarrow$ 4.4% / 13.0%	binding
peak decode rate	$\pm 30\%$	none (0 of 19,342 decode calls bound)	non-binding
cache bytes per token	$\pm 30\%$	none (peak HBM near 39% of capacity)	non-binding

class priority, long flows simply never receive a device slot while interactive traffic saturates the cluster, and the result is locked by a regression test. A delta against zero is undefined, which is why this table reports absolute rates. Section 2 argued that a flow can be under budget and starved at the same time, and that a scheduler can be locally optimal at every step while starving the flow; Section 4.2 argued that priorities are not contracts; this table is both arguments, measured. FCFS, blind to class, completes 0.332 and degrades 5.0 points under injection: no isolation, no containment, but no starvation either. The discipline is the only policy in the set that is simultaneously above 0.3 absolute and below 3.5 points of adversarial degradation.

7.5 Ablations: which element carries the cost

The ablation campaign removes one element at a time from the tuned discipline, nine variants at three mixes and three seeds, at two operating points: an uncontended nominal-load cell, and a contended cell with four times the decode slots, 1.3x load, and the grid’s shortest gaps, chosen because the binding analysis of Section 7.6 shows the memory and peak-decode constraints demonstrably engage there. We report the campaign’s result as its own noise analysis forces us to: across both operating points, no element of the discipline shows an effect on the primary metric distinguishable from seed noise. At the contended cell, the total goodput spread across all nine variants is 0.68 percent, against a within-variant seed spread whose median is 29 percent; at the nominal cell the two spreads are the same order of magnitude. One apparent effect appeared and did not survive: at the nominal cell, replacing headroom-keyed degradation with queue-depth keying improved goodput by 20.9 percent, but the effect vanished at the contended cell (+0.2 percent) and sits within the nominal cell’s own seed spread; an earlier internal readout called it corroborating evidence, and that reading is withdrawn in the released, corrected document. With three seeds, a per-element median is a draw from a distribution wider than the effects being measured, and this design cannot separate an inert element from an unexercised one; per-element attribution needs swept load and slot axes and materially more replications, and the per-run data is released for exactly that. This noise floor is a property of the ablation design, not of the primary comparison: H1’s per-cell deltas are paired, every policy scoring byte-identical workload realizations, which cancels the workload draw that dominates the unpaired within-variant spread here, and the H1 verdict is additionally invariant to interval readings, since zero cells reach the support threshold under any of them.

What the campaign does establish is robust because it is large and consistent rather than a difference between near-identical numbers: reserved capacity sits 88 to 95 percent idle at both operating points, despite the contended cell’s quadrupled slots, elevated load, and minimal gaps. The reason is structural. An agentic flow runs a short invocation and then waits in tool time, a duty cycle of roughly 5 percent by construction, so capacity reserved for such flows is stranded nine parts in ten regardless of configuration, and no variant in either cell recovers the loss. The ablation campaign’s honest conclusion is that the mechanism’s cost is not located in any single element; it is located in the duty cycle of the workload the mechanism reserves for. Section 4.2 presented the forward-looking commitment signal as a design claim; at the operating points tested here, no advantage of that signal over queue depth is attributable above the noise floor, and the system-level H2 comparison of Section 7.3 is negative. The tuning campaign’s independent, cross-family K-to-floor result (Section 5), the optimizer choosing the smallest reserved subset offered in every workload family at both horizons, points the same way at larger scale.

7.6 Sensitivity: what the numbers are conditioned on

Service-model constants are literature-anchored estimates, and the evaluation measured which ones matter (Table 4). The single-stream decode rate is the binding constant, and it is material: at 0.7x the anchored

value, the representative cell’s deficit compresses from 21.5 to 4.4 percent; at 1.3x it is 13.0 percent. The sign of the verdict is robust across the tested range; the magnitude is genuinely uncertain at roughly the factor the anchor is uncertain, and we state this as the paper’s principal fidelity caveat, unresolvable here because the hardware phase was gated on this evaluation’s outcome and the gate closed. Two further constants, peak decode rate and cache bytes per token, do not move the representative cell’s result at all, and the released binding analysis shows why, with instrumentation rather than assertion: at 16 decode slots the batch is capped below the size at which peak decode can bind (0 of 19,342 decode calls), and peak memory occupancy stays near 39 percent of capacity, so a 30 percent perturbation changes no placement decision. At a deliberately memory-bound configuration (64 slots, agentic-heavy, 1.5x load), both constants demonstrably bind (peak decode governs 43 percent of calls, memory saturates), confirming the harness detects binding where binding exists and that the main grid’s device configuration sits in the compute-bound regime. Regression tests pin all three properties.

8 The horizon lesson

The evaluation’s most generalizable finding is about measurement, and it was found the way such things are found: by investigating a suspicious zero. The starvation diagnostic of Section 7.4 revealed, alongside the genuine starvation, that every sweep in the campaign was running a measurement horizon far shorter than the workload’s own timescale. The median agentic flow requires roughly 711 seconds to complete with no queueing at all, tool-time gaps plus decode service; the figure was first measured at roughly 863 seconds on the diagnostic realization that surfaced the problem, and the released generator derivation, which any reader can run, gives 711. The build’s default horizons ran 150 to 300 seconds, and at a 200-second window only about a quarter of agentic flows (24.5 percent of the population; 21.5 percent of the diagnostic realization) could complete on an idle cluster. The primary metric counts completed flows. A window shorter than the unit of value does not measure the metric; it measures truncation.

The bias is not symmetric, and it points at completion-guaranteeing policies specifically. A reservation pays its stranding cost during the flow and collects its payoff at completion; a horizon that ends before completion charges the full cost against zero payoff, the mechanism’s worst case by construction, while policies that never schedule long flows at all are barely touched. Quantified on one core cell (Figure 5), the discipline’s deficit against the best baseline was 22.3 percent at a 300-second horizon and 7.8 percent at 1500 seconds, stable through 6000 seconds; at the longest horizon the discipline completed more flows in absolute terms than its comparator while still losing on value-weighted goodput, a real loss, but half the reported one.

The subtler effect is that censoring corrupts tuning, not only measurement. Re-tuning every policy at the corrected horizon flipped selected parameters across the board, most cleanly for the multi-level feedback baseline, whose chosen quantum moved from the shortest value in its grid to the longest, in every workload family. Under a censored horizon, aggressive demotion looks good because long flows are truncated before their cost lands; correct the horizon and the same tuner, on the same grid, chooses patience. An evaluation that had tuned at 300 seconds and measured at 1500 would have compared the mechanism against baselines mis-tuned in its favor. Both tuning campaigns are released.

The fix is structural, not vigilant: the released code asserts, before any sweep runs, that the configured horizon exceeds the workload’s derivable median flow lifetime, and the verdict path independently rejects any manifest whose horizon disagrees with the grid it claims to report. The correction was adopted as a numbered amendment before the final grid ran, under a one-fix clause barring further measurement changes in either direction, and its full history, including that the correction improved the mechanism’s number on the probe cell and worsened it on H2, is in the released amendment log. The rule generalizes to any completion-metric evaluation of long-horizon agents, a category currently multiplying: horizons must exceed the completable-lifetime distribution of the workload, or the benchmark measures its own window. We found the cost of honoring this rule to be severe, superlinear in the horizon under sustained overload, and it forced a disclosed reduction of our own grid; we suspect the rule is widely violated for exactly this reason.

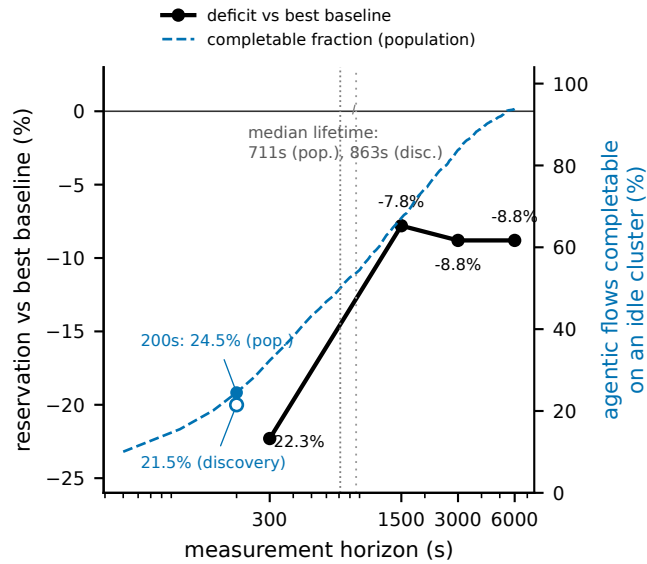


Figure 5: Horizon censoring on one core cell. The population median agentic flow lifetime, derived from the released workload generators, is roughly 711 seconds (first measured at 863 seconds on the diagnostic realization that surfaced the artifact, open markers); at a 200-second window only about a quarter of agentic flows can complete even on an idle cluster (24.5 percent of the population; 21.5 percent of the diagnostic realization), while the campaign’s default horizons ran 150 to 300 seconds. The measured deficit is 22.3 percent at a 300-second horizon and halves to 7.8 percent at adequate horizons, stable through 6000 seconds. Horizons shorter than the workload’s completable lifetimes charge guarantee-holders full stranding cost against zero completion payoff (Section 8).

9 Discussion

The ledger, in one place. Completion guarantees cost a median 10.3 percent of goodput per provisioned GPU-hour in the tested region, and the cost is U-shaped in load: deepest, 22 to 24 percent, exactly at nominal provisioning, recovering toward saturation, inverting past it. They buy three measured things: interactive tail-latency isolation of 63 to 97 percent across most of the region, with one identified inversion corner; containment of adversarial runaways to a 3.1-point degradation where the strongest request-oriented schedulers starve compliant agentic work to zero; and a regime boundary at 1.5x load beyond which the guarantee begins to pay for itself. The efficiency thesis, that the purchases outweigh the cost on a single hardware-normalized metric, is dead on its own pre-registered terms. What survives is a price list.

The verdict closes each of Section 3’s transfers on its own terms. Lesson one’s economics largely do not survive the crossing: the field that invented per-flow guarantees abandoned them substantially for their cost, and this evaluation measured the same abandonment logic operating at the serving layer, with the added force that the tuner, offered a range of reserved-subset sizes under an honest metric, chose the smallest in every workload family at every horizon. The mechanism, optimized truthfully, minimizes itself; the duty-cycle analysis of Section 7.5 says why, and says it is structural rather than configurational. Lesson two’s shape survives but its transferred signal does not: degradation should still act early, probabilistically, and per class, but keying it to committed headroom rather than queue state, this design’s one deliberate departure from the RED lineage, delivered no measurable advantage at the system level (Section 7.3) and no attributable one at the component level (Section 7.5), and the forward signal this paper argued the serving layer gets for free is, on this evidence, worth nothing that queue depth does not already provide. Lessons three and four cross intact, because they were never economic claims: fate-independent supervision and provenance-bound admission are authority structures, their cost is engineering rather than goodput, and the one property of theirs this evaluation could measure, the budget’s containment of adversarial runaways, held exactly as specified while the strongest priority schedulers starved every compliant flow they were nominally serving. The contract-versus-priority distinction, argued in Section 4.2 as a matter of kind, appears in Section 7.4 as a matter of record.

The honest reading of the mechanism, then, is not “efficiency technique that failed” but “insurance product whose premium is now known.” Fleets that are well-provisioned and run nominal load sit at the bottom of the U and should not reserve: the premium is maximal there, and the starvation pathology, while real, is avoidable with cheaper instruments at low agentic share. The case for reservations concentrates where the purchases concentrate: fleets that price interactive tail latency highly and currently buy isolation through overprovisioning; fleets with adversarial or untrusted agentic exposure, where containment is not replicable by per-request caps; and fleets that run hot, past saturation, where the sign has already flipped. Even there, the K-to-floor result disciplines enthusiasm: reserve for the few flows whose completion is worth a known premium, and for nothing else, which is what Section 3’s first lesson prescribed before the evaluation priced it.

Three things could move the verdict, and we name them rather than imply them. Hardware calibration: the binding sensitivity of Section 7.6 means the deficit’s magnitude, not its sign, is uncertain at roughly the uncertainty of one anchored constant; a calibrated rerun is mechanical if the constant is ever measured. Workload shape: gap structure drives stranding, and agentic workloads with tighter tool-time distributions than our anchors would shrink the premium. Objective: this evaluation priced everything into one number by design; an operator who prices tail latency or adversarial robustness as first-class objectives is solving a different optimization, and Section 7’s decomposition is written to let them price it. What cannot move the verdict is re-measurement under this registration: the one-fix clause spent its fix, and the number stands.

10 Limitations

Simulation only. The hardware phase was pre-gated on this evaluation supporting the mechanism, and the gate closed; every number here is conditioned on the simulator’s service model. The model’s constants are literature-anchored, marked low confidence, and their measured sensitivity is the paper’s principal caveat: the binding constant moves the representative deficit between 4.4 and 21.5 percent, sign-stable throughout.

Synthetic workloads: no public traces of long-horizon agentic serving existed at specification freeze; generators, anchors, and seeds are released, and the adversarial family is constructed, not observed. Three seeds per cell (Amendment 3): intervals are wider at low load, and several 0.7x cells have upper bounds crossing zero individually, though the verdict is invariant to any reading of them, since zero cells reach the support threshold under any interpretation. Cluster sizes 8 to 32: the 64-device axis was removed for measured infeasibility at the corrected horizon; the trend across measured sizes is flat, and the memory-bound regime is probed at one configuration rather than swept. Value weights are operator-supplied, fixed at registration, and swept in sensitivity. H2 and H3 are measured on representative configurations, not crossed with the full grid. The two-cell ablation design cannot separate any element’s effect from its measured seed-noise floor at three replications; the campaign attributes the mechanism’s cost to the workload’s structural duty cycle, and finer per-element attribution requires swept axes and more seeds (data released). Finally, the registration’s one-fix clause means any residual measurement artifact, in either direction, stands uncorrected in these numbers by design; the amendment log is the complete record of what was corrected and when.

11 Related work

The serving-layer families were treated in Section 2; here we fix the distinctions. Four of the six baselines implement the scheduling family this paper evaluates against directly: priority preemption with paged cache (Kwon et al., 2023), multi-level feedback (Wu et al., 2023), deadline-class demotion (Goel et al., 2025), and AIMD admission with pause/resume (Chen et al., 2026); migration and chunked-prefill scheduling (Sun et al., 2024; Agrawal et al., 2024) are compatible mechanics not separately evaluated. The MARS-style baseline (Wang et al., 2026) represents external admission control for agentic serving; its dynamic SLO goodput is the nearest metric relative, differing in unit and denominator as Section 5 states, and its cost-benefit cache retention is the advisory counterpart of the contractual pinning evaluated here, alongside time-to-live and estimate-driven retention (Li et al., 2025; Abhyankar et al., 2024). Goodput-oriented routing for agentic inference over heterogeneous pools (Du et al., 2026) pursues the same completion-timeliness objective by placement and migration, within the preemption-family philosophy this evaluation tests against, and reports gains orthogonal to the admission question studied here. Program-level schedulers order agentic work without resource contracts and compose beneath either approach (Luo et al., 2025); workflow-aware serving (Wadlom et al., 2026), OS-inspired agent process management (Agyemang et al., 2026), and client-side cost-ladder overload control (Yuan et al., 2026) are likewise compatible and orthogonal. Gateway budget enforcement (LiteLLM, 2026; Wang, 2026) supplies the cumulative accounting the discipline’s budgets extend from ceiling to contract; account-level provisioned throughput (Amazon Web Services, 2026; Microsoft Corporation, 2026) reserves at a granularity that cannot see flows, and composes as the tenant envelope of Section 4.2. Batch-system advance reservations, quality-of-service classes, and gang scheduling (SchedMD LLC, 2026; The Kubernetes Authors, 2026; Ousterhout, 1982) are the structural ancestors; what is new here is what was tested, jointly guaranteed cache-and-budget reservations across idle gaps at flow granularity, and the finding is that the ancestors’ economics survive the transfer better than the guarantees do. The network lineage of Section 3 (Braden et al., 1997; Blake et al., 1998; Floyd & Jacobson, 1993; Heinanen et al., 1999; Ferguson & Senie, 2000; Baker & Savola, 2004; Intel Corporation et al., 2013) is inheritance, not reargued here.

On the trust side, capability and information-flow systems (Debenedetti et al., 2025; Costa et al., 2025; Zhong et al., 2025), trained instruction hierarchies (Wallace et al., 2024), their formally enforced descendants (Fan et al., 2026; Maloyan & Namiot, 2026), and execution-time integrity layers (Santos-Grueiro, 2026) govern what content may cause an agent to do, downstream of admission; Section 4.4’s element is upstream and complementary, governing what capacity and standing a flow’s provenance may hold, with trust-thresholded admission previously appearing for one resource, an agent’s memory store (Wu et al., 2026). On the monitoring side, in-pipeline guards and representation-level interventions (Inan et al., 2023; Zou et al., 2024) supply verdicts; execution-time safety kernels hold an out-of-reach stop at the action layer (Dobrin & Chmiel, 2026), and halt mechanisms reachable through the model’s input channel inherit that channel’s attack surface (Lee et al., 2025); Section 4.3’s claim sits beside these as an authority and availability structure under which any verdict remains enforceable. The plane-separation convergence (Gupta, 2026; NVIDIA Corporation, 2025; Shan et al., 2025; Wang et al., 2026) is this paper’s premise. On method, pre-registration with published kill criteria remains rare in systems evaluation; the horizon result of Section 8 relates to right-censoring

in survival analysis and to warmup discipline in queueing simulation, applied to a benchmark category, completion-metric agent evaluation, that is currently multiplying.

12 Conclusion

This paper proposed a completion-oriented resource contract for the serving layer, built it from four lessons packet networking paid for, stated in advance and in public the numbers that would prove its economics wrong, and then measured them. The economics are wrong: a median 10.3 percent loss, no supporting cell in 108, the graceful-degradation hypothesis reversed, and the cause structural, a workload whose 5 percent duty cycle strands whatever is reserved for it. What the contract provides regardless, isolation, containment, an audit record, and a stop that the computation being stopped cannot reach, is measured or holds by construction, and is now priced rather than free. The starvation of compliant agentic work by request-oriented schedulers is no longer an argument but a table. The evaluation leaves one rule for the benchmarks being built around long-horizon agents: the horizon must exceed the workload’s completable lifetimes, or the benchmark measures its own window. Every artifact required to check, retune, or overturn these numbers is released.

Artifacts

The release contains: the evaluation specification frozen 2026-07-06 with its complete amendment log and integrity rulings; the simulator (engine, seven policies behind one interface, workload generators, 141 tests); the tuning harness with both tuning campaigns (censored and corrected horizons, all tuned parameters and scores); all run manifests including the quarantined pre-amendment set and the collapsed-run records; the preliminary readouts with superseded banners intact; the final readout and the analysis code that computes the verdict as a pure function of the manifest set; figure-generation code with one command per figure; and per-figure reproduction commands. Nothing reported in this paper depends on an unreleased artifact.

A Admission test and ledger invariants

State, per pool p : reserved set R_p with bound K_p ; capacity vector Cap_p (compute, device memory, spill-tier memory and restore bandwidth); forecast window W ; $committed(p, t) = \text{sum over } r \text{ in } R_p \text{ of } r\text{'s remaining claim profile over } [t, t+W]$; per-class budget quotas Q_c ; per-tenant envelopes E_g .

ADMIT(f) for flow f with descriptor (class c , tenant g , budget b , footprint m , profile est):

1. $|R_p| < K_p$
 2. $committed(p, t) + claim(est) \leq ceiling(p)$ for all t in window
 3. exists device set D : $pinnable(m, D)$ under embodiment (strict: resident throughout; tiered: resident-active plus reserved spill and reserved restore within bound $L_{restore}$)
 4. $spent(c) + b \leq Q_c$ and within E_g
 5. $trust(f) \geq \text{eligibility threshold for reservation}$
- All pass: grant $r(f)$, append to ledger, $R_p += \{f\}$.
Any fail: route to class c best-effort, or decline with optional quote.

Events: RENEW is ADMIT at current state with no carried privilege. EXPIRE and EXHAUST release $r(f)$ with the class’s disposition. HALT(authority) releases $r(f)$ immediately; it is the only release not initiated by the flow’s own lifecycle.

Invariants (any interleaving of admit, renew, expire, exhaust, halt):

- I1 Compute: $committed(p, t) \leq ceiling(p) \leq Cap_p$ at all t .
- I2 Memory: pinned bytes + reserved spill/restore never exceed their

tiers; a tiered-pin flow always has a reserved restore path.

- I3 Budget conservation: metered decode tokens of f never exceed b ; quota accounting is exact under renegotiation.
- I4 Guarantee monotonicity: once granted, $r(f)$ is removed only by expire, exhaust, or halt; no capacity condition appears in any release path.

B Degradation probability families

$p(H)$ requirements: p monotone non-increasing in H ; $p(H) = 0$ for $H \geq H_{\max}$; $p(H) = p_{\text{ceiling}}$ for $H \leq H_{\min}$. Families: piecewise-linear between $(H_{\max}, 0)$ and $(H_{\min}, p_{\text{ceiling}})$; exponential $p = p_{\text{ceiling}} * ((H_{\max} - H)/(H_{\max} - H_{\min}))^k$ for shaping early gentleness; per-class and per-provenance parameterization as a matrix $\{H_{\max}, H_{\min}, p_{\text{ceiling}}, \text{action ladder}\}_{c, tr}$ with lower classes and lower trust assigned earlier $H_{\max}$ and higher p_{ceiling} . H computed from the ledger over forecast window W ; W trades responsiveness against stability and is swept in the evaluation.

C The transfer table, expanded

1. Rationed guarantees. Origin: per-flow reservations failed at scale; edge-complexity with core aggregates survived (Braden et al., 1997; Blake et al., 1998). Serving gap: no per-task guarantee exists at any scale. Element: bounded reserved subset K over aggregate classes (Section 4). Leak: K 's right size is workload-dependent and swept, not derived; the evaluation's tuner chose the smallest offered (Section 7.5). 2. Early probabilistic class-aware shedding. Origin: (Floyd & Jacobson, 1993; Heinanen et al., 1999). Serving gap: deterministic cliffs (demotion at deadline, rejection at ceiling). Element: $p(H)$ ladder (Section 4.2). Departure, not transfer: the signal is ledger commitments, a forward quantity networks lacked. Leak: forecast quality bounds the signal's advantage, and the evaluation found no advantage attributable above its noise floor (Sections 7.3, 7.5). 3. Fate-independent supervision. Origin: out-of-band management (Intel Corporation et al., 2013). Serving gap: oversight in the supervised pipeline. Element: management plane with authority-and-availability independence (Section 4.3). Leak: signal dependence is retained and conceded. 4. Provenance-bound authority at the edge. Origin: ingress source validation (Ferguson & Senie, 2000; Baker & Savola, 2004). Serving gap: content adjudicated downstream can command capacity. Element: trust-keyed reservation eligibility (Section 4.4). Leak: provenance classes are coarser than network paths, and class assignment itself must be trustworthy.

D Per-cell H1 results

Per-cell H1 results, machine-generated from the FINAL readout (cell, best baseline, improvement, 95 percent CI, guardrail; all 108 cells).

cell	best baseline	improvement	95% CI	guardrail
mix0.1_load0.7_dev16_gap20.0	fcfs	-7.8%	[-7.8%, +9.6%]	ok
mix0.1_load0.7_dev16_gap300.0	fcfs	-6.9%	[-6.9%, +9.1%]	ok
mix0.1_load0.7_dev16_gap60.0	fcfs	-7.3%	[-7.3%, +9.3%]	ok
mix0.1_load0.7_dev32_gap20.0	fcfs	-7.3%	[-7.7%, +9.9%]	ok
mix0.1_load0.7_dev32_gap300.0	fcfs	-6.6%	[-6.7%, +9.2%]	ok
mix0.1_load0.7_dev32_gap60.0	fcfs	-7.0%	[-7.2%, +9.5%]	ok
mix0.1_load0.7_dev8_gap20.0	fcfs	-8.5%	[-8.5%, +8.5%]	ok
mix0.1_load0.7_dev8_gap300.0	fcfs	-7.8%	[-7.8%, +7.9%]	ok
mix0.1_load0.7_dev8_gap60.0	fcfs	-8.1%	[-8.1%, +8.0%]	ok
mix0.1_load1.0_dev16_gap20.0	vllm_style	-23.2%	[-23.2%, -3.7%]	ok
mix0.1_load1.0_dev16_gap300.0	vllm_style	-23.2%	[-23.2%, -3.3%]	ok
mix0.1_load1.0_dev16_gap60.0	vllm_style	-23.2%	[-23.2%, -3.6%]	ok
mix0.1_load1.0_dev32_gap20.0	vllm_style	-23.5%	[-23.5%, -4.1%]	ok
mix0.1_load1.0_dev32_gap300.0	vllm_style	-23.5%	[-23.5%, -3.4%]	ok

cell	best baseline	improvement	95% CI	guardrail
mix0.1_load1.0_dev32_gap60.0	vllm_style	-23.5%	[-23.5%, -3.6%]	ok
mix0.1_load1.0_dev8_gap20.0	vllm_style	-22.5%	[-22.7%, -3.3%]	ok
mix0.1_load1.0_dev8_gap300.0	fastserve_mlfq	-22.6%	[-22.7%, -5.0%]	FAIL (+77.2%)
mix0.1_load1.0_dev8_gap60.0	vllm_style	-22.6%	[-22.7%, -3.0%]	ok
mix0.1_load1.3_dev16_gap20.0	niyama_style	-10.6%	[-10.9%, +9.2%]	ok
mix0.1_load1.3_dev16_gap300.0	niyama_style	-10.7%	[-11.1%, +9.5%]	ok
mix0.1_load1.3_dev16_gap60.0	niyama_style	-10.7%	[-11.0%, +9.7%]	ok
mix0.1_load1.3_dev32_gap20.0	niyama_style	-10.6%	[-10.7%, +9.1%]	ok
mix0.1_load1.3_dev32_gap300.0	niyama_style	-10.8%	[-10.9%, +10.1%]	ok
mix0.1_load1.3_dev32_gap60.0	niyama_style	-10.7%	[-10.8%, +9.9%]	ok
mix0.1_load1.3_dev8_gap20.0	niyama_style	-10.0%	[-10.7%, +9.1%]	ok
mix0.1_load1.3_dev8_gap300.0	niyama_style	-10.2%	[-10.8%, +9.2%]	ok
mix0.1_load1.3_dev8_gap60.0	niyama_style	-10.1%	[-10.7%, +8.8%]	ok
mix0.1_load1.5_dev16_gap20.0	niyama_style	+2.2%	[+1.5%, +19.7%]	ok
mix0.1_load1.5_dev16_gap300.0	niyama_style	+2.0%	[+1.3%, +20.3%]	ok
mix0.1_load1.5_dev16_gap60.0	niyama_style	+2.1%	[+1.4%, +20.3%]	ok
mix0.1_load1.5_dev32_gap20.0	niyama_style	+1.8%	[+1.6%, +19.5%]	ok
mix0.1_load1.5_dev32_gap300.0	niyama_style	+1.7%	[+1.5%, +20.5%]	ok
mix0.1_load1.5_dev32_gap60.0	niyama_style	+1.7%	[+1.6%, +20.2%]	ok
mix0.1_load1.5_dev8_gap20.0	niyama_style	+2.5%	[+1.9%, +20.0%]	ok
mix0.1_load1.5_dev8_gap300.0	niyama_style	+2.3%	[+1.8%, +20.9%]	ok
mix0.1_load1.5_dev8_gap60.0	niyama_style	+2.4%	[+1.8%, +20.6%]	ok
mix0.3_load0.7_dev16_gap20.0	fcfs	-10.9%	[-11.2%, +10.9%]	ok
mix0.3_load0.7_dev16_gap300.0	fcfs	-7.9%	[-8.0%, +9.7%]	ok
mix0.3_load0.7_dev16_gap60.0	fcfs	-9.2%	[-9.4%, +10.7%]	ok
mix0.3_load0.7_dev32_gap20.0	fcfs	-10.5%	[-11.0%, +11.0%]	ok
mix0.3_load0.7_dev32_gap300.0	fcfs	-7.4%	[-7.7%, +9.4%]	ok
mix0.3_load0.7_dev32_gap60.0	fcfs	-8.9%	[-9.2%, +10.6%]	ok
mix0.3_load0.7_dev8_gap20.0	fcfs	-11.0%	[-11.6%, +10.4%]	ok
mix0.3_load0.7_dev8_gap300.0	fcfs	-8.6%	[-8.7%, +8.7%]	ok
mix0.3_load0.7_dev8_gap60.0	fcfs	-9.7%	[-10.1%, +10.1%]	ok
mix0.3_load1.0_dev16_gap20.0	vllm_style	-23.3%	[-23.3%, -7.9%]	ok
mix0.3_load1.0_dev16_gap300.0	vllm_style	-23.3%	[-23.3%, -4.1%]	ok
mix0.3_load1.0_dev16_gap60.0	vllm_style	-23.3%	[-23.3%, -5.3%]	ok
mix0.3_load1.0_dev32_gap20.0	vllm_style	-23.4%	[-23.5%, -7.8%]	ok
mix0.3_load1.0_dev32_gap300.0	vllm_style	-23.5%	[-23.6%, -4.6%]	ok
mix0.3_load1.0_dev32_gap60.0	vllm_style	-23.5%	[-23.6%, -5.7%]	ok
mix0.3_load1.0_dev8_gap20.0	vllm_style	-22.4%	[-22.9%, -7.8%]	ok
mix0.3_load1.0_dev8_gap300.0	vllm_style	-22.4%	[-22.9%, -3.9%]	ok
mix0.3_load1.0_dev8_gap60.0	vllm_style	-22.4%	[-22.9%, -5.0%]	ok
mix0.3_load1.3_dev16_gap20.0	niyama_style	-10.5%	[-10.8%, +4.9%]	ok
mix0.3_load1.3_dev16_gap300.0	niyama_style	-11.0%	[-11.3%, +7.0%]	ok
mix0.3_load1.3_dev16_gap60.0	niyama_style	-10.8%	[-11.0%, +6.3%]	ok
mix0.3_load1.3_dev32_gap20.0	niyama_style	-10.5%	[-10.7%, +4.6%]	ok
mix0.3_load1.3_dev32_gap300.0	niyama_style	-11.1%	[-11.2%, +8.2%]	ok
mix0.3_load1.3_dev32_gap60.0	niyama_style	-10.8%	[-10.9%, +7.0%]	ok
mix0.3_load1.3_dev8_gap20.0	niyama_style	-9.9%	[-10.5%, +4.1%]	ok
mix0.3_load1.3_dev8_gap300.0	niyama_style	-10.4%	[-11.0%, +7.1%]	ok
mix0.3_load1.3_dev8_gap60.0	niyama_style	-10.1%	[-10.8%, +5.8%]	ok
mix0.3_load1.5_dev16_gap20.0	niyama_style	+2.3%	[+1.8%, +15.4%]	ok
mix0.3_load1.5_dev16_gap300.0	niyama_style	+1.7%	[+1.1%, +19.1%]	ok
mix0.3_load1.5_dev16_gap60.0	niyama_style	+2.0%	[+1.4%, +17.6%]	ok
mix0.3_load1.5_dev32_gap20.0	niyama_style	+2.0%	[+2.0%, +14.9%]	ok
mix0.3_load1.5_dev32_gap300.0	niyama_style	+1.4%	[+1.3%, +18.9%]	ok
mix0.3_load1.5_dev32_gap60.0	niyama_style	+1.7%	[+1.6%, +17.4%]	ok
mix0.3_load1.5_dev8_gap20.0	niyama_style	+3.2%	[+2.1%, +15.1%]	ok
mix0.3_load1.5_dev8_gap300.0	niyama_style	+2.6%	[+1.5%, +19.1%]	ok
mix0.3_load1.5_dev8_gap60.0	niyama_style	+2.9%	[+1.7%, +17.1%]	ok
mix0.5_load0.7_dev16_gap20.0	fastserve_mlfq	-16.0%	[-16.3%, -5.4%]	ok
mix0.5_load0.7_dev16_gap300.0	fcfs	-9.5%	[-9.8%, +10.3%]	ok
mix0.5_load0.7_dev16_gap60.0	vllm_style	-12.5%	[-12.9%, +1.5%]	ok
mix0.5_load0.7_dev32_gap20.0	fastserve_mlfq	-16.1%	[-16.3%, -5.2%]	ok
mix0.5_load0.7_dev32_gap300.0	fcfs	-9.4%	[-9.4%, +10.4%]	ok
mix0.5_load0.7_dev32_gap60.0	fcfs	-12.5%	[-12.5%, +9.7%]	ok
mix0.5_load0.7_dev8_gap20.0	vllm_style	-10.0%	[-15.9%, -4.7%]	ok
mix0.5_load0.7_dev8_gap300.0	fcfs	-9.8%	[-9.8%, +8.8%]	ok

cell	best baseline	improvement	95% CI	guardrail
mix0.5_load0.7_dev8_gap60.0	fcfs	-12.8%	[-12.8%, +9.2%]	ok
mix0.5_load1.0_dev16_gap20.0	vllm_style	-22.8%	[-23.0%, -13.2%]	ok
mix0.5_load1.0_dev16_gap300.0	vllm_style	-23.3%	[-23.5%, -6.6%]	ok
mix0.5_load1.0_dev16_gap60.0	vllm_style	-23.2%	[-23.3%, -10.1%]	ok
mix0.5_load1.0_dev32_gap20.0	vllm_style	-23.0%	[-23.1%, -13.4%]	ok
mix0.5_load1.0_dev32_gap300.0	vllm_style	-23.6%	[-23.7%, -6.6%]	ok
mix0.5_load1.0_dev32_gap60.0	vllm_style	-23.4%	[-23.5%, -10.0%]	ok
mix0.5_load1.0_dev8_gap20.0	vllm_style	-22.6%	[-22.6%, -13.5%]	ok
mix0.5_load1.0_dev8_gap300.0	vllm_style	-23.0%	[-23.0%, -6.2%]	ok
mix0.5_load1.0_dev8_gap60.0	vllm_style	-22.9%	[-22.9%, -9.3%]	ok
mix0.5_load1.3_dev16_gap20.0	niyama_style	-10.1%	[-10.4%, -2.7%]	ok
mix0.5_load1.3_dev16_gap300.0	niyama_style	-11.4%	[-11.6%, +4.0%]	ok
mix0.5_load1.3_dev16_gap60.0	niyama_style	-10.8%	[-11.1%, +0.8%]	ok
mix0.5_load1.3_dev32_gap20.0	niyama_style	-10.2%	[-10.4%, -2.2%]	ok
mix0.5_load1.3_dev32_gap300.0	niyama_style	-11.5%	[-11.7%, +5.2%]	ok
mix0.5_load1.3_dev32_gap60.0	niyama_style	-11.0%	[-11.1%, +1.5%]	ok
mix0.5_load1.3_dev8_gap20.0	niyama_style	-9.5%	[-10.0%, -3.5%]	ok
mix0.5_load1.3_dev8_gap300.0	niyama_style	-10.8%	[-11.4%, +4.5%]	ok
mix0.5_load1.3_dev8_gap60.0	niyama_style	-10.3%	[-10.8%, +1.0%]	ok
mix0.5_load1.5_dev16_gap20.0	niyama_style	-1.5%	[-2.2%, +6.2%]	FAIL (+501.7%)
mix0.5_load1.5_dev16_gap300.0	niyama_style	+1.1%	[+0.6%, +15.6%]	ok
mix0.5_load1.5_dev16_gap60.0	niyama_style	+1.9%	[+1.2%, +11.7%]	ok
mix0.5_load1.5_dev32_gap20.0	niyama_style	-1.4%	[-1.9%, +6.9%]	FAIL (+468.6%)
mix0.5_load1.5_dev32_gap300.0	niyama_style	+0.8%	[+0.7%, +15.9%]	ok
mix0.5_load1.5_dev32_gap60.0	niyama_style	+1.5%	[+1.4%, +11.9%]	ok
mix0.5_load1.5_dev8_gap20.0	niyama_style	-1.7%	[-1.9%, +6.1%]	FAIL (+526.5%)
mix0.5_load1.5_dev8_gap300.0	niyama_style	+1.5%	[+1.0%, +14.7%]	ok
mix0.5_load1.5_dev8_gap60.0	niyama_style	+2.2%	[+1.7%, +11.5%]	ok

References

- Reyna Abhyankar, Zijian He, Vikranth Srivatsa, Hao Zhang, and Yiyang Zhang. InferCept: Efficient intercept support for augmented large language model inference. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pp. 81–95, 2024.
- Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. arXiv preprint arXiv:2403.02310, 2024.
- Justice Owusu Agyemang, Jerry John Kponyo, Obed Kwasi Somuah, Elliot Amponsah, Godfred Manu Addo Boakye, and Kwame Opuni-Boachie Obour Agyekum. HiveMind: OS-inspired scheduling for concurrent LLM agent workloads. arXiv preprint arXiv:2604.17111, 2026.
- Amazon Web Services. Increase model invocation capacity with provisioned throughput in Amazon Bedrock. Amazon Bedrock User Guide, 2026. URL <https://docs.aws.amazon.com/bedrock/latest/userguide/prov-throughput.html>. Accessed 2026-07-24.
- Fred Baker and Pekka Savola. Ingress filtering for multihomed networks. RFC 3704, BCP 84, RFC Editor, March 2004. URL <https://www.rfc-editor.org/info/rfc3704>.
- Steven Blake, David Black, Mark Carlson, Elwyn Davies, Zheng Wang, and Walter Weiss. An architecture for differentiated services. RFC 2475, Informational, RFC Editor, December 1998. URL <https://www.rfc-editor.org/info/rfc2475>.
- Robert Braden, Lixia Zhang, Steven Berson, Shai Herzog, and Sugih Jamin. Resource ReSerVation protocol (RSVP) – version 1 functional specification. RFC 2205, Proposed Standard, RFC Editor, September 1997. URL <https://www.rfc-editor.org/info/rfc2205>.
- Qiaoling Chen, Zhisheng Ye, Tian Tang, Peng Sun, Boyu Tian, Guoteng Wang, Shenggui Li, Yonggang Wen, Zhenhua Han, and Tianwei Zhang. CONCUR: High-throughput agentic batch inference of LLM via congestion-based concurrency control. arXiv preprint arXiv:2601.22705, 2026.

- Manuel Costa, Boris Köpf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. Securing AI agents with information-flow control. arXiv preprint arXiv:2505.23643, 2025.
- Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. Defeating prompt injections by design. arXiv preprint arXiv:2503.18813, 2025.
- Seth Dobrin and Łukasz Chmiel. The unfireable safety kernel: Execution-time AI alignment for AI agents and other escapable AI systems. arXiv preprint arXiv:2606.26057, 2026.
- Boxiao Du, Boning Huangfu, Yizhou Luo, Chen Chen, Zijun Li, Minchen Yu, Xiaoyi Fan, and Minyi Guo. GoodServe: Towards high-goodput serving of agentic LLM inferences over heterogeneous resources. arXiv preprint arXiv:2605.16867, 2026.
- European Parliament and Council of the European Union. Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act), article 14. Official Journal of the European Union, OJ L, 2024/1689, 12.7.2024, CELEX 32024R1689, 2024. URL <https://eur-lex.europa.eu/eli/reg/2024/1689/oj>.
- EvalSpec. Pre-registered evaluation specification and amendment log for reservation-based serving (EVALSPEC release). Supplementary material accompanying this submission, 2026. Frozen 2026-07-06; Amendments 1-3, clarification, and integrity rulings included (EVALSPEC_RELEASE.md).
- Linfeng Fan, Ziwei Li, Yuan Tian, Yichen Wang, Rongsheng Li, and Xiong Wang. The granularity mismatch in agent security: Argument-level provenance solves enforcement and isolates the LLM reasoning bottleneck. arXiv preprint arXiv:2605.11039, 2026.
- Paul Ferguson and Daniel Senie. Network ingress filtering: Defeating denial of service attacks which employ IP source address spoofing. RFC 2827, BCP 38, RFC Editor, May 2000. URL <https://www.rfc-editor.org/info/rfc2827>.
- Sally Floyd and Van Jacobson. Random early detection gateways for congestion avoidance. *IEEE/ACM Transactions on Networking*, 1(4):397–413, August 1993.
- Kanishk Goel, Jayashree Mohan, Nipun Kwatra, Ravi Shreyas Anupindi, and Ramachandran Ramjee. Niyama: Breaking the silos of LLM inference serving. arXiv preprint arXiv:2503.22562, 2025.
- Anushri Gupta. What is an LLM control plane? Mozilla.ai Blog, June 2026. URL <https://blog.mozilla.ai/what-is-an-llm-control-plane/>. Published 2026-06-12.
- Juha Heinanen, Fred Baker, Walter Weiss, and John Wroclawski. Assured forwarding PHB group. RFC 2597, Proposed Standard, RFC Editor, June 1999. URL <https://www.rfc-editor.org/info/rfc2597>.
- Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabisa. Llama guard: LLM-based input-output safeguard for human-AI conversations. arXiv preprint arXiv:2312.06674, 2023.
- Intel Corporation, Hewlett-Packard Company, NEC Corporation, and Dell Inc. Intelligent platform management interface specification, second generation, v2.0, revision 1.1. Industry specification, 2013. URL <https://www.intel.com/content/dam/www/public/us/en/documents/product-briefs/ipmi-second-gen-interface-spec-v2-rev1-1.pdf>.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.
- Sechan Lee, Hyoungun Kim, and Sangdon Park. Can we stop malicious AI? KILLBENCH: A benchmark for external AI kill switch feasibility. arXiv preprint arXiv:2511.13725, 2025.

- Hanchen Li, Runyuan He, Qiuyang Mang, Qizheng Zhang, Huanzhi Mao, Xiaokun Chen, Hangrui Zhou, Alvin Cheung, Joseph Gonzalez, and Ion Stoica. Continuum: Efficient and robust multi-turn LLM agent scheduling with KV cache time-to-live. arXiv preprint arXiv:2511.02230, 2025.
- LiteLLM. Agent iteration budgets. LiteLLM documentation, 2026. URL https://docs.litellm.ai/docs/a2a_iteration_budgets. Accessed 2026-07-24; per-session iteration and dollar budgets enforced at the gateway.
- Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E. Gonzalez, and Ion Stoica. Autellix: An efficient serving engine for LLM agents as general programs. arXiv preprint arXiv:2502.13965, 2025.
- Narek Maloyan and Dmitry Namiot. Sleeper channels and provenance gates: Persistent prompt injection in always-on autonomous AI agents. arXiv preprint arXiv:2605.13471, 2026.
- Microsoft Corporation. Provisioned throughput for foundry models. Microsoft Learn documentation, 2026. URL <https://learn.microsoft.com/en-us/azure/foundry/openai/concepts/provisioned-throughput>. Accessed 2026-07-24; provisioned throughput units (PTUs).
- NVIDIA Corporation. NVIDIA Dynamo: Overall architecture. Dynamo documentation, v1.3.0, 2025. URL <https://docs.dynamo.nvidia.com/dynamo/design-docs/overall-architecture>. Accessed 2026-07-24.
- John K. Ousterhout. Scheduling techniques for concurrent systems. In *Proceedings of the Third International Conference on Distributed Computing Systems (ICDCS)*, pp. 22–30, 1982.
- Igor Santos-Grueiro. Context-to-execution integrity for LLM agents. arXiv preprint arXiv:2607.06000, 2026.
- SchedMD LLC. Slurm workload manager: Advanced resource reservation guide. Slurm documentation, version 26.05, 2026. URL <https://slurm.schedmd.com/reservations.html>. Accessed 2026-07-24.
- Jiaxin Shan, Varun Gupta, Le Xu, Haiyang Shi, Jingyuan Zhang, Ning Wang, Linhui Xu, Rong Kang, Tongping Liu, Yifei Zhang, Yiqing Zhu, Shuwei Jin, Gangmuk Lim, Binbin Chen, Zuzhi Chen, Xiao Liu, Xin Chen, Kante Yin, Chak-Pong Chung, Chenyu Jiang, Yicheng Lu, Jianjun Chen, Caixue Lin, Wu Xiang, Rui Shi, and Liguang Xie. AIBrix: Towards scalable, cost-effective large language model inference infrastructure. arXiv preprint arXiv:2504.03648, 2025.
- Zhaoyuan Su, Zeyu Zhang, Tingfeng Lan, Zirui Wang, Haiying Shen, Juncheng Yang, and Yue Cheng. MorphServe: Efficient and workload-aware LLM serving via runtime quantized layer swapping and KV cache resizing. arXiv preprint arXiv:2506.02006, 2025.
- Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. Llumnix: Dynamic scheduling for large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.
- The Kubernetes Authors. Pod quality of service classes. Kubernetes documentation, 2026. URL <https://kubernetes.io/docs/concepts/workloads/pods/pod-qos/>. Accessed 2026-07-24.
- Noppanat Wadlom, Junyi Shen, and Yao Lu. Efficient LLM serving for agentic workflows: A data systems perspective. arXiv preprint arXiv:2603.16104, 2026.
- Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. The instruction hierarchy: Training LLMs to prioritize privileged instructions. arXiv preprint arXiv:2404.13208, 2024.
- Boyu Wang. Governing multi-agent systems: Agent identity, A2A, and the agent gateway. TrueFoundry Blog, June 2026. URL <https://www.truefoundry.com/blog/multi-agent-a2a-governance-gateway>. Published 2026-06-19.

Yifei Wang, Hancheng Ye, Yechen Xu, Cong Guo, Chiyue Wei, Qinsi Wang, Dongting Li, Tingjun Chen, Hai Li, Danyang Zhuo, and Yiran Chen. MARS: Efficient, adaptive co-scheduling for heterogeneous agentic systems. arXiv preprint arXiv:2604.26963, 2026.

Beining Wu, Zihao Ding, Jun Huang, and Yanxiao Zhao. Forget to improve: On-device LLM-agent continual learning via budget-curated memory. arXiv preprint arXiv:2606.25115, 2026.

Bingyang Wu, Yinmin Zhong, Zili Zhang, Shengyu Liu, Fangyue Liu, Yuanhang Sun, Gang Huang, Xuanzhe Liu, and Xin Jin. Fast distributed inference serving for large language models. arXiv preprint arXiv:2305.05920, 2023.

Renzhong Yuan, Yijun Zeng, Xiaosong Gao, Linxi Yu, Haochun Liao, and Han Wang. Scheduling the unschedulable: Taming black-box LLM inference at scale. arXiv preprint arXiv:2604.06970, 2026.

Peter Yong Zhong, Siyuan Chen, Ruiqi Wang, McKenna McCall, Ben L. Titzer, Heather Miller, and Phillip B. Gibbons. RTBAS: Defending LLM agents against prompt injection and privacy leakage. arXiv preprint arXiv:2502.08966, 2025.

Andy Zou, Long Phan, Justin Wang, Derek Duenas, Maxwell Lin, Maksym Andriushchenko, Rowan Wang, Zico Kolter, Matt Fredrikson, and Dan Hendrycks. Improving alignment and robustness with circuit breakers. arXiv preprint arXiv:2406.04313, 2024.